Git Folder Structure Fix Guide

A step-by-step guide to fixing messy GitHub folder structure after separate pushes

---

1. The Problem

In one repository (Capstone-Project), the folder structure pushed to GitHub was correct: the **part1** and **part2** folders appeared exactly as they exist locally in VS Code, each containing their own files.

In another repository (keshav), the GitHub page showed only **README.md** and **main.py** sitting loose at the root, even though the local VS Code project clearly had **part1** and **part2** folders, each with its own files inside.

How this happened:

- The **part1** folder was created first and pushed to GitHub on its own.
- Later, the **part2** folder was created and pushed separately.
- Because of this, Git did not track both folders together under one clean parent structure — likely because a separate **.git** folder existed inside one of the subfolders, or each folder was pushed as if it were its own repository.

The core lesson: a Git repository should be initialized **once**, at the top-level project folder — not separately inside each subfolder.

2. How to Diagnose the Issue

Before fixing anything, check whether a stray **.git** folder exists inside part1 or part2. This is the most common cause of this exact problem.

Open a terminal at the parent folder (the one containing both part1 and part2):

```
cd C:\Users\kesha\OneDrive\Desktop\keshav
```

Check each subfolder for a hidden .git directory:

```
dir /a part1 dir /a part2
```

A **.git** folder should exist only once, at the top level of the project (e.g. keshav\.git). If it also appears inside part1 or part2, that is the bug.

3. Step-by-Step Fix

Step 1 – Go to the parent folder

Navigate to the top-level project folder that contains both part1 and part2.

```
cd C:\Users\kesha\OneDrive\Desktop\keshav
```

Step 2 – Remove any stray .git folders inside subfolders

If Step 2 in the diagnosis found a .git folder inside part1 or part2, delete it. This prevents Git from treating the subfolder as its own separate repository.

```
rmdir /s /q part1\.git
rmdir /s /q part2\.git
```

Step 3 – Initialize Git at the top level only

This creates a single .git folder at the root of the project.

```
git init
```

Step 4 – Check what Git sees

Confirm that both part1/ and part2/ appear as folders to be tracked. If a folder does not show up here, check for .gitignore rules excluding it.

```
git status
```

Step 5 – Stage and commit everything

This adds the full, correct folder structure to Git in one commit.

```
git add .
git commit -m "Add part1 and part2 with correct folder structure"
```

Step 6 – Connect to the existing GitHub repository

Link the local project to the GitHub repo. If a remote is already set, use set-url instead of add.

```
git remote add origin https://github.com/Keshav30-jpg/keshav.git

# If it says remote already exists, use this instead:
git remote set-url origin https://github.com/Keshav30-jpg/keshav.git
```

Step 7 – Force push the corrected structure

Since the old GitHub history is messy, force-push the clean local structure to overwrite it.

```
git branch -M main
git push -u origin main --force
```

Step 8 – Verify on GitHub

Refresh the GitHub repository page. part1/ and part2/ should now appear as folders, matching the correct structure seen in the Capstone-Project repo.

> Warning: **--force** overwrites whatever is currently on GitHub with the local folder structure. Only use it if the old commit history on GitHub is not needed.

4. How to Avoid This in the Future

- Run **git init** only **once**, at the very top folder of the project — never inside individual subfolders like part1 or part2.
- When a new folder is added later (e.g. part3), simply run **git add .** from the top-level folder. Do not cd into the subfolder and run git init there.
- Before pushing, always run **git status** first to confirm every folder that should be included actually shows up as tracked or staged.
- Periodically check for stray .git folders inside subdirectories using **dir /a foldername** (Windows) if the project structure looks wrong on GitHub.

5. Quick Command Reference

Command	Purpose
git init	Initialize a new Git repository in the current folder
git status	Show tracked, untracked, and staged files/folders
git add .	Stage all files and folders in the current directory
git commit -m "message"	Save staged changes with a description
git remote add origin <url>	Link the local repo to a GitHub repository
git remote set-url origin <url>	Change an existing remote's URL
git branch -M main	Rename the current branch to 'main'
git push -u origin main	Push local commits to GitHub and set upstream
git push -u origin main --force	Push and overwrite GitHub history with local version
rmdir /s /q foldername\.git	(Windows) Delete a stray .git folder inside a subfolder